

DINORESERVER

SISTEMA ERP DE GESTIÓN PARA PARQUE TEMÁTICO JURÁSICO



Gestión de Dinosaurios +

Gestión de Recintos +

Empleados, Inventario y Mantenimiento y Reportes

Realizado por:
AYMAN BAKHOUSA



DinoReserve

— E R P —

Sistema ERP de Gestión para Parque Temático Jurásico

Tabla de contenido

1. Briefing	3
Funcionalidades Principales	4
Consideraciones Técnicas	4
2. Concepto	5
DinoReserve ERP — Jurassic Command Center	5
Paleta de Colores del Sistema	5
3. Justificación del Proyecto	6
Motivación Técnica	6
Relevancia Académica	6
Proyección Profesional	6
4. Tecnologías Utilizadas	7
Python 3.10+	7
Tkinter / ttk	7
SQLite3	7
Visual Studio Code	7
Herramientas de documentación	7
5. Ingeniería del Software	8
5.1 Arquitectura MVC por Capas	8
5.2 Pseudocódigo del Programa	8
5.3 Estructura de la Base de Datos	9
5.4 Módulos del Sistema	9
6. Dificultades Encontradas	11
7. Análisis DAFO	12
8. Documentación — Manual de Usuario	13
8.1 Requisitos del Sistema	13
8.2 Instalación y Arranque	13
8.3 Guía de Uso por Módulo	13
9. Proyección — Líneas de Investigación Futuras	14
10. Conclusiones y Valoración Personal	15
10.1 Conclusiones Técnicas	15
10.2 Objetivos Cumplidos	15
10.3 Valoración Personal	15
11. Bibliografía	16

I. Briefing

Objetivo: Desarrollar una aplicación ERP de escritorio en Python para la gestión integral de DinoReserve, un parque temático de dinosaurios. El sistema centraliza la administración de recintos, especies, empleados, taquilla, inventario, mantenimiento y análisis operativo.

Funcionalidades Principales

- **Gestión de Dinosaurios:** CRUD completo con control de especie, dieta, nivel de peligro (1–5), estado de salud y nivel de alimentación. Asignación validada a recintos.
- **Gestión de Recintos:** Control de capacidad, tipo de zona, voltaje de valla eléctrica y nivel de seguridad.
- **Taquilla:** Venta de tickets con tipos Normal / Infantil / VIP / Fast Pass, cálculo de IVA al 21 % y generación de código localizador único.
- **Empleados:** Registro con rol, salario, nivel técnico 1–5, zona asignada y estado (activo / vacaciones / inactivo).
- **Inventario:** Control de stock con alertas de mínimo, categorías y gestión de proveedores.
- **Mantenimiento:** Tareas con prioridad codificada por color, asignación de empleado y seguimiento de estado.
- **Reportes:** KPIs agregados con SQL (JOIN, GROUP BY, SUM, AVG), gráfico de barras en canvas y exportación CSV.
- **Motor de Eventos Aleatorios:** Simulación probabilística de incidencias del parque con persistencia en base de datos.
- **Logs del Sistema:** Registro persistente de actividad con niveles INFO / WARNING / ERROR y filtros en tiempo real.

Consideraciones Técnicas

- Python 3.10+ — zero dependencias externas.
- Tkinter / ttk para interfaz gráfica de escritorio multiplataforma.
- SQLite3 integrado para persistencia relacional con 8 tablas relacionadas.
- Arquitectura MVC desacoplada en capas: models / repositories / services / controllers / views.
- Sistema de login con roles y hashing SHA-256 de contraseñas.
- Logging propio persistente en archivo con niveles de severidad.

II. Concepto

DinoReserve ERP — Jurassic Command Center

DinoReserve: es un sistema ERP de escritorio desarrollado íntegramente en Python, concebido como solución integral para la gestión operativa de un parque temático de dinosaurios. El proyecto combina una interfaz visual de alto impacto inspirada en estéticas HUD futuristas con una arquitectura de software sólida y académicamente rigurosa.

La temática jurásica define la paleta de colores, la nomenclatura de los módulos (Parque, Recintos, Taquilla) y la lógica de negocio: niveles de peligro, voltaje de vallas, motor de eventos aleatorios que simula incidencias reales del parque.

Paleta de Colores del Sistema

Token	Hex	Uso en la interfaz
COLOR_BG	#06110B	Fondo principal — verde noche
COLOR_PRIMARY	#35E879	Elementos activos, bordes, texto primario
COLOR_ACCENT	#D9A441	Títulos, KPIs destacados, dorado fósil
COLOR_DANGER	#FF5D5D	Alertas críticas, acciones destructivas
COLOR_SUCCESS	#4DE6A2	Confirmaciones, estados saludables
COLOR_TEXT	#F1FFF5	Texto general sobre fondos oscuros

III. Justificación del Proyecto

La elección de DinoReserve como Proyecto Final de DAW responde a tres motivaciones: demostrar el dominio técnico de Python y la POO, crear una aplicación de escritorio visualmente impactante, y construir un sistema que sirva como referencia real de arquitectura de software.

Motivación Técnica

El proyecto explora en profundidad la arquitectura MVC aplicada a aplicaciones de escritorio con Tkinter. Se demuestra que Tkinter, frecuentemente subestimado, puede producir interfaces modernas y complejas con un diseño de componentes bien estructurado, uso extensivo de canvas y un sistema de temas centralizado.

Relevancia Académica

- Cumple todos los requisitos del currículo DAW: POO con herencia y polimorfismo, persistencia relacional con SQLite, SQL agregado (JOIN, GROUP BY, SUM, AVG).
- Implementa excepciones personalizadas y manejo robusto de errores.
- Incorpora un sistema de logging persistente por niveles conforme al enunciado.
- La base de datos contiene 8 tablas relacionadas, superando el mínimo exigido de 5.
- Los datos de demostración son narrativamente coherentes con la temática jurásica.

Proyección Profesional

La temática jurásica es un vehículo creativo que permite diseñar lógica de negocio compleja (motor de eventos probabilísticos, control de capacidad, alertas de stock, cálculo de IVA) en un contexto memorable. El resultado es un proyecto de portfolio que demuestra capacidad de diseño de sistemas, no solo habilidad de codificación.

IV. Tecnologías Utilizadas

Python 3.10+

Lenguaje principal del proyecto. Python ofrece una sintaxis clara, una librería estándar completa y excelente soporte para POO. La decisión de usar únicamente la librería estándar, sin dependencias externas, garantiza que la aplicación funcione en cualquier instalación estándar sin pasos adicionales de configuración.

Motivo de elección: portabilidad total y ecosistema compatible con el stack objetivo de data analyst.

Tkinter / ttk

Framework de interfaz gráfica incluido en Python. Permite construir aplicaciones de escritorio multiplataforma (Windows, macOS, Linux). Se combina con ttk para widgets modernizados y un sistema de estilos personalizable mediante ttk.Style. El canvas se usa extensivamente para efectos HUD y gráficos de reportes.

SQLite3

Motor de base de datos relacional integrado en Python. Ideal para aplicaciones de escritorio gracias a su naturaleza serverless. El esquema implementa 8 tablas con claves foráneas, índices de rendimiento y restricciones CHECK para integridad de datos.

Tablas: users, enclosures, dinosaurs, employees, tickets, inventory, maintenance_tasks, random_events.

Visual Studio Code

Editor de código utilizado para el desarrollo. Ofrece resaltado de sintaxis para Python, depuración integrada y control de versiones con Git.

Extensiones clave: Python, Pylance, SQLite Viewer, GitLens.

Herramientas de documentación

Para la memoria del proyecto se han empleado herramientas de maquetación profesional que permiten combinar texto, imágenes y elementos visuales en documentos de alta calidad. La paleta y estilo visual de la memoria reflejan la identidad corporativa de DinoReserve.

El logo fue diseñado con IA generativa.

V. Ingeniería del Software

En este apartado se describe el proceso de diseño y desarrollo del sistema DinoReserve: la arquitectura elegida, los patrones aplicados, la estructura de base de datos y el funcionamiento de cada módulo.

5.1 Arquitectura MVC por Capas

DinoReserve implementa el patrón Modelo-Vista-Controlador de forma estricta, separando responsabilidades en capas independientes:

Capa	Carpeta	Responsabilidad
Models	models/	Clases de datos (DTOs). Representan entidades del dominio.
Repositories	repositories/	Acceso a datos. Encapsulan todas las queries SQL.
Services	services/	Lógica de negocio. Orquestan repositorios y aplican reglas.
Controllors	controllors/	Coordinan vistas y servicios. Gestionan eventos UI.
Views	views/	Interfaz gráfica Tkinter/ttk. Sin lógica de negocio.
Utils	utils/	Constantes, validadores, logger, cargador de assets.
Database	database/	Conexión SQLite, inicialización del esquema y migración.

5.2 Pseudocódigo del Programa

INICIO DinoReserve ERP

inicializar_base_de_datos()
SI esquema no existe → ejecutar schema.sql
SI existe BD antigua → migrar al nuevo nombre

BUCLE PRINCIPAL:
mostrar LoginView()
SI usuario cierra ventana → SALIR
verificar credenciales → hash SHA-256
SI credenciales válidas:
mostrar DashboardView(usuario)
SI logout explícito → volver al BUCLE
SI cierra ventana → SALIR

MÓDULO Taquilla:
seleccionar tipo de ticket (Normal/Infantil/VIP/Fast Pass)
calcular precio base + IVA 21 %
generar código localizador único
registrar venta en tabla tickets

MÓDULO Reportes:
consultar tickets por periodo (JOIN + GROUP BY + SUM)
renderizar gráfico de barras en canvas Tkinter
exportar datos a CSV

FIN

5.3 Estructura de la Base de Datos

La base de datos SQLite contiene 8 tablas relacionadas mediante claves foráneas. El PRAGMA foreign_keys = ON garantiza la integridad referencial en tiempo de ejecución.

Tabla	Campos clave	Relaciones
users	id, username, password_hash, role	—
enclosures	id, name, zone_type, max_capacity, fence_voltage, security_level	← dinosaurs, employees, tasks
dinosaurs	id, name, species, diet_type, danger_level, health_status, enclosure_id	→ enclosures
employees	id, name, role, salary, assigned_zone, technical_level, status	→ enclosures
tickets	id, visitor_name, ticket_type, price, iva, total, locator_code	—
inventory	id, item_name, category, quantity, minimum_stock, supplier	—
maintenance_tasks	id, asset_name, priority, assigned_employee_id, status	→ employees
random_events	id, event_type, description, severity, affected_id	—

5.4 Módulos del Sistema

Dashboard — Command Center

Pantalla principal tras el login. Muestra sidebar con navegación activa, topbar con reloj en vivo, muro de KPIs del parque, panel de ocupación de recintos y accesos rápidos a cada módulo.

Dinosaurios

CRUD completo. KPI cards en cabecera (total, por dieta, enfermos). Filtros por dieta y estado de salud. Formulario modal validado para altas y ediciones. Control de asignación de recinto con validación de capacidad.

Recintos

Control de capacidad en tiempo real. KPIs por tipo de zona. Filtro por estado (activo / mantenimiento / cerrado). Edición de voltaje de valla y nivel de seguridad.

Taquilla

Interfaz de venta con radio-cards por tipo de ticket. Preview en tiempo real del precio e IVA. Código localizador único. Historial de ventas con filtro por fecha.

Empleados

KPIs de personal (activos, veterinarios, en vacaciones, nómina total). Filtros por rol y estado. Asignación a zona del parque. Escala visual de nivel técnico 1–5.

Inventario

Control de stock con alertas para items bajo mínimo. Categorías por tipo de producto. Filtro por categoría y modo de stock crítico. Acciones rápidas de reposición.

Mantenimiento

Tareas con prioridad (crítica / alta / media / baja) codificadas por color. Asignación a empleados. Seguimiento de estado (pendiente / en curso / completado / cancelado).

Reportes

KPIs agregados con SQL (JOIN, GROUP BY, SUM, AVG). Gráfico de barras en canvas Tkinter. Exportación a CSV.

Eventos Aleatorios

Motor probabilístico con 6 tipos de eventos (tormenta, fallo de valla, alerta sanitaria...). Severidades: info / warning / critical. Persistencia de todos los eventos en base de datos.

Logs del Sistema

Visor del archivo log_parque.txt con filtros por texto y nivel de severidad. Copia al portapapeles. Actualización en tiempo real.

VI. Dificultades Encontradas

- **Gestión del ciclo de vida de ventanas Tkinter**

Tkinter no está diseñado nativamente para el patrón login → dashboard → login. La solución fue implementar un bucle principal en main.py que instancia y destruye ventanas de forma controlada, detectando si el cierre fue por logout explícito o por cierre de ventana.

- **Sistema de assets con fallback seguro**

El logo y los fondos son opcionales. Se implementó un cargador de imágenes con soporte condicional para Pillow (calidad superior) y degradación elegante a tk.PhotoImage si no está disponible, sin bloquear el arranque de la aplicación.

- **Migración de base de datos entre versiones**

Entre versiones del proyecto el archivo de BD fue renombrado. Se implementó detección automática del archivo legacy y migración transparente al nuevo nombre al arrancar la aplicación.

- **Motor de eventos probabilísticos**

Diseñar un motor que genere eventos con probabilidades predefinidas (suma = 1.0) y los persista sin duplicados en reinicios requirió combinar random.choices() con lógica de negocio en la capa de servicios.

- **Consistencia visual multiplataforma**

Tkinter renderiza de forma diferente en Windows y macOS. Centralizar todos los tokens visuales en constants.py y theme.py permitió ajustar un único punto para conseguir coherencia entre plataformas.

- **Rendimiento de tablas con datos grandes**

Los widgets Treeview de Tkinter pueden volverse lentos con muchas filas. Se implementó paginación y límites en las consultas SQL, con índices de rendimiento en las columnas más consultadas.

VII. Análisis DAFO

FORTALEZAS	DEBILIDADES
• Zero dependencias externas: instalación instantánea. • Arquitectura MVC limpia y completamente desacoplada. • Diseño visual premium y coherente con identidad propia. • Base de datos con integridad referencial y datos de demo completos. • Sistema de logging y motor de eventos — diferenciadores únicos.	• Interfaz limitada al entorno de escritorio. • Tkinter tiene un techo de diseño inferior a frameworks web. • Sin sistema de backups automáticos de la BD. • Login de un solo factor, sin recuperación de contraseña. • Reportes limitados a exportación CSV.
OPORTUNIDADES	AMENAZAS
• Migración futura a PyQt6 / CustomTkinter para UI más moderna. • Portado a web con Flask/Django manteniendo la lógica de negocio. • Integración de API REST para sincronización con apps móviles. • Base para proyectos de análisis de datos con pandas/matplotlib. • Plantilla reutilizable para ERPs de otros sectores.	• Frameworks web dominan el desarrollo empresarial actual. • SQLite no escala a entornos multiusuario concurrentes. • Tkinter pierde soporte activo frente a PyQt o CustomTkinter. • Posibles incompatibilidades en actualizaciones futuras de Python.

VIII. Documentación — Manual de Usuario

8.1 Requisitos del Sistema

- Python 3.10 o superior instalado en el sistema.
- Sistema operativo: Windows 10/11, macOS 12+ o Linux con entorno de escritorio.
- Tkinter incluido en la distribución de Python (viene por defecto en CPython).
- Sin necesidad de instalar paquetes adicionales (requirements.txt vacío).
- Espacio en disco: ~10 MB para la aplicación y la base de datos.

8.2 Instalación y Arranque

1. Descomprimir el proyecto

Extraer dinoreserve.zip en la ubicación deseada.

El directorio raíz debe contener main.py y las carpetas del proyecto.

2. Entorno virtual (opcional)

python -m venv .venv

Windows: .venv\Scripts\activate

macOS/Linux: source .venv/bin/activate

3. Ejecutar la aplicación

python main.py

La base de datos se inicializa automáticamente en el primer arranque.

4. Credenciales de demo

Administrador: admin / admin123

Operador: operador / operador123

8.3 Guía de Uso por Módulo

- **Login:** Introducir usuario y contraseña. Usar el icono de ojo para mostrar u ocultar la contraseña. El rol del usuario (admin / operador) determina los permisos de acceso.
- **Dashboard:** Pantalla principal que muestra el estado global del parque. Los KPI cards se actualizan en cada visita. Usar el sidebar izquierdo para navegar entre módulos. El botón de logout se encuentra en la parte inferior del sidebar.
- **Agregar un Dinosaurio:** Módulo Dinosaurios → botón «+ Nuevo». Rellenar el formulario modal: nombre, especie, tipo de dieta, nivel de peligro (1-5), estado de salud y recinto asignado. La capacidad del recinto se valida automáticamente.
- **Vender un Ticket:** Módulo Taquilla → seleccionar tipo de ticket con los radio-cards. Ingresar nombre del visitante. El precio e IVA se calculan en tiempo real. Confirmar para registrar la venta y generar el código localizador.
- **Generar un Reporte:** Módulo Reportes → seleccionar periodo de fecha. El sistema ejecuta las consultas SQL y renderiza el gráfico. Botón «Exportar CSV» para descargar los datos tabulados.
- **Disparar un Evento:** Módulo Eventos → botón «Generar Evento». El motor selecciona aleatoriamente un tipo según probabilidades configuradas. El evento queda registrado con timestamp y severidad. Filtrar por severidad con el selector superior.

IX. Proyección — Líneas de Investigación Futuras

DinoReserve v2.0 es una base sólida sobre la que se identifican múltiples líneas de evolución técnica y funcional:

1. Migración a interfaz web (Flask + React)

Portar la lógica de negocio a una API REST con Flask manteniendo los servicios y repositorios existentes. Construir un frontend moderno en React o Vue.js. Esta migración aprovecharía el alineamiento con el stack del Proyecto Final Valore (Flask + PostgreSQL + JS).

2. Sustitución de SQLite por PostgreSQL

Para soporte multiusuario concurrente, migrar a PostgreSQL usando SQLAlchemy como ORM. El esquema relacional actual es completamente compatible con PostgreSQL.

3. Módulo de análisis de datos

Integrar pandas y matplotlib para análisis estadístico de ventas, ocupación de recintos y salud del inventario. Directamente alineado con la orientación a Data Analyst del autor.

4. Notificaciones y alertas en tiempo real

Implementar un sistema de alertas para stock crítico, fallos de mantenimiento urgentes y eventos de alta severidad mediante threading de Tkinter o websockets web.

5. Autenticación avanzada

Añadir 2FA, recuperación de contraseña por email y permisos granulares por módulo y acción (lectura, escritura, eliminación) por rol de usuario.

6. Exportación a PDF nativa

Integrar ReportLab para generar informes de ventas, fichas de dinosaurio y reportes de mantenimiento directamente desde la interfaz de la aplicación.

7. Aplicación móvil companion

Diseñar una app React Native o Flutter que consuma la API REST futura para consulta del estado del parque desde dispositivos móviles.

X. Conclusiones y Valoración Personal

10.1 Conclusiones Técnicas

DinoReserve ERP demuestra que Python y Tkinter son herramientas capaces de producir aplicaciones de escritorio robustas, visualmente impactantes y académicamente rigurosas. La arquitectura MVC por capas ha facilitado el desarrollo iterativo y el mantenimiento del código, permitiendo añadir módulos sin afectar a los existentes.

La decisión de usar exclusivamente la biblioteca estándar de Python ha sido simultáneamente un reto y un logro: forzó soluciones creativas para efectos visuales (canvas HUD), gestión de imágenes con fallback y animaciones, resultando en código más comprensible y portable.

10.2 Objetivos Cumplidos

- Arquitectura MVC desacoplada con herencia y polimorfismo en vistas y componentes.
- Base de datos relacional con 8 tablas, claves foráneas y restricciones de integridad.
- SQL agregado implementado: JOIN, GROUP BY, SUM, AVG en el módulo de Reportes.
- Sistema de logging persistente por niveles con filtros en tiempo real.
- Motor de eventos aleatorios con probabilidades configurables y persistencia.
- Interfaz premium con identidad visual propia y coherente en todos los módulos.
- Sistema de login con hashing SHA-256 y roles diferenciados.
- Datos de demostración completos y narrativamente coherentes con la temática.

10.3 Valoración Personal

Este proyecto representa mi mayor esfuerzo técnico hasta la fecha en el ciclo DAW. Partir de una idea temática original y llevarla hasta un ERP funcional con diez módulos, interfaz premium y arquitectura profesional ha sido un proceso intenso y enormemente satisfactorio.

Los mayores aprendizajes han sido en diseño de sistemas: cómo separar responsabilidades, cómo hacer que un componente visual sea reutilizable y cómo gestionar el estado de una aplicación de escritorio sin un framework que lo haga automáticamente. Estas habilidades son directamente transferibles al desarrollo web y al análisis de datos.

DinoReserve no es solo un proyecto académico: es una demostración de que es posible construir software con personalidad, que sea a la vez funcional, bien estructurado y visualmente memorable.

XI. Bibliografía

Documentación oficial de Python 3.10

<https://docs.python.org/3.10/>

Referencia principal del lenguaje, librería estándar, Tkinter y sqlite3.

Tkinter 8.5 Reference: A GUI for Python

<https://anzelg.github.io/rin2/book2/2405/docs/tkinter/>

Manual de referencia de widgets, canvas, eventos y ttk.

SQLite Documentation

<https://www.sqlite.org/docs.html>

Referencia de SQL, PRAGMA, índices y tipos de datos en SQLite.

Real Python — Tkinter Tutorials

<https://realpython.com/python-gui-tkinter/>

Tutoriales prácticos de Tkinter y ttk avanzado.

Refactoring Guru — MVC Pattern

<https://refactoring.guru/design-patterns>

Referencia de patrones de diseño MVC y Repository.

PEP 8 — Style Guide for Python Code

<https://pep8.org/>

Guía de estilo de código Python seguida en el proyecto.